

TechTribers

Java Interview Series · Part 1 · Q01–Q74

Core Java Interview Questions

Fundamentals, control flow and object-oriented programming, answered the way a senior engineer would answer them.

74	63	12	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `CJ-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

74	11	63	12
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

☕	01	Java Fundamentals	Q01–Q09	5
🧮	02	Variables, Data Types & Operators	Q10–Q14	12
🔀	03	Control Flow	Q15–Q22	17
🧩	04	Coding Programs	Q23–Q24	23
🏗️	05	Classes, Objects & OOP Pillars	Q25–Q35	25
🧰	06	Constructors & Object Basics	Q36–Q41	34
🎭	07	Polymorphism, Abstraction & Interfaces	Q42–Q50	39
🪆	08	Nested & Inner Classes	Q51–Q55	46
🔐	09	final, static, this & super	Q56–Q66	51
🗒️	10	Object Methods & Initializers	Q67–Q72	58
🧭	11	Inheritance Edge Cases	Q73–Q74	64

© TechTribers. For personal use only. Do not redistribute.

All questions

☕ Java Fundamentals

Q01	What is Java?	5
Q02	Briefly describe the history of Java.	5
Q03	What features make Java a popular programming language?	6
Q04	Compare C++ and Java: differences and similarities.	7
Q05	Write a simple "Hello, World!" program.	7
Q06	What is the structure of a Java program, and how does it run?	8
Q07	How do you set the path for Java?	9
Q08	Differentiate between JDK, JRE and JVM.	10
Q09	Describe the role and purpose of the JVM.	10

🧮 Variables, Data Types & Operators

Q10	What are variables in Java? How are they declared and used?	12
Q11	Discuss the data types available in Java with examples.	13
Q12	How does Java handle Unicode characters, and why does it matter?	14
Q13	Explain the types of operators in Java with examples.	15
Q14	List some important keywords in Java and their significance.	16

🔀 Control Flow

Q15	What is the purpose of the if-else statement? Give an example.	17
Q16	Explain the switch statement and provide an example.	18
Q17	What is the purpose of the for loop? Provide an example.	19
Q18	Explain the while loop and provide an example.	20
Q19	Describe the do-while loop. Provide an example.	20
Q20	What is the purpose of the break statement? Give an example.	21
Q21	Explain the continue statement and provide an example.	21
Q22	Describe the different types of comments in Java.	22

🧩 Coding Programs

Q23	Write a Java program to find the sum of all even numbers from 1 to 100.	23
Q24	Write a program to check whether a number is a palindrome.	23

🏗️ Classes, Objects & OOP Pillars

Q25	What is a class?	25
Q26	What is an object?	26
Q27	What is the difference between a class and an object?	27
Q28	What are instance variables?	27
Q29	What are instance methods?	28
Q30	What is encapsulation?	28
Q31	What is inheritance?	29
Q32	What is the `super` keyword used for?	30
Q33	What is method overriding?	31
Q34	Can you override a private or final method?	32
Q35	What is method overloading?	33

🧰 Constructors & Object Basics

Q36	What is a constructor?	34
Q37	Can you have multiple constructors in a class?	35
Q38	What is the default constructor?	35
Q39	What is the purpose of the `this` keyword?	36
Q40	What is the difference between a shallow copy and a deep copy?	37
Q41	What is the `instanceof` operator used for?	38

© TechTribers. For personal use only. Do not redistribute.

🎭 Polymorphism, Abstraction & Interfaces

Q42	What is polymorphism?	39
Q43	What is an abstract class?	40
Q44	Can an abstract class have a constructor?	41
Q45	What is an interface?	41
Q46	What is the difference between an abstract class and an interface?	42
Q47	Can an interface have variables?	43
Q48	What is a default method in an interface?	43
Q49	Can an interface extend another interface?	45
Q50	What is a static method in an interface?	45

🪆 Nested & Inner Classes

Q51	What is a nested class?	46
Q52	What is a static nested class?	47
Q53	What is a local inner class?	48
Q54	What is an anonymous inner class?	48
Q55	What is an inner class, and what are its advantages?	49

🔐 final, static, this & super

Q56	What is a final class?	51
Q57	What is a final method?	52
Q58	What is a final variable?	52
Q59	What is a static variable?	53
Q60	What is the `final` keyword used for?	54
Q61	What is the `static` keyword used for?	54
Q62	Can `this` be used in a static context, and what is constructor chaining?	55
Q63	What are the rules for calling `super(...)` in a constructor?	55
Q64	How do you prevent a class from being inherited?	56
Q65	Which methods cannot be overridden, and how do you prevent overriding?	57
Q66	Can a class be both `abstract` and `final`?	57

🗒️ Object Methods & Initializers

Q67	What is the purpose of the `toString()` method?	58
Q68	What is the `equals()` method used for?	59
Q69	What is the `hashCode()` method used for?	59
Q70	What is the purpose of the `clone()` method?	61
Q71	What is the instance initializer block used for?	62
Q72	What is the static initializer block used for?	63

🧭 Inheritance Edge Cases

Q73	How do you achieve multiple inheritance in Java?	64
Q74	What is method hiding in Java?	65

© TechTribers. For personal use only. Do not redistribute.

Section 01

Java Fundamentals

What Java is, where it came from, and how a .java file becomes running machine code.

Questions 1–9 · 9 questions



Q01 CJ-001 • Basic

What is Java?

⚡ Quick answer

A statically typed, object-oriented, garbage-collected language that compiles to bytecode and runs on the JVM, so the same build runs on any OS.

Java was created by James Gosling's team at Sun Microsystems and released in 1995. Oracle has owned it since acquiring Sun in 2010, and the open-source reference implementation is **OpenJDK**.

Three properties define it in interviews:

- **Compiled to bytecode**, not native code. The JVM interprets it and JIT-compiles hot paths to native code at runtime.
- **Managed memory.** Objects live on a garbage-collected heap; there is no manual `free`.
- **Strong static typing** with a huge standard library and ecosystem (Spring, Kafka, Hadoop, Android).

💡 Pro tip

Say "write once, run anywhere" and then immediately explain *why*: bytecode plus a platform-specific JVM. That second sentence is what separates a fresher answer from a solid one.

Q02 CJ-002 • Basic

Briefly describe the history of Java.

⚡ Quick answer

Started as "Oak" at Sun in 1991, released as Java 1.0 in 1995, acquired by Oracle in 2010, and on a 6-month release cadence since 2017 with an LTS every two years.

The milestones interviewers actually care about:

- **Java 5 (2004)**: generics, annotations, enums, autoboxing, enhanced for-loop, `java.util.concurrent`.
- **Java 8 (2014)**: lambdas, Streams API, `Optional`, default methods, `java.time`.

© TechTribers. For personal use only. Do not redistribute.

- **Java 9 (2017)**: module system (JPMS), JShell. After this, a new release every March and September.
- **Java 11 (2018)**: first LTS of the new cadence; `var` in lambdas, standard `HttpClient`.
- **Java 17 (2021)**: records, sealed classes, pattern matching for `instanceof`, text blocks.
- **Java 21 (2023)**: virtual threads, record patterns, pattern matching for `switch`, sequenced collections.
- **Java 25 (2025)**: the current LTS.



⚠️ **Interview trap**

Oracle acquired Sun in **2010** (the deal closed in January 2010), not 2011. Small, but interviewers notice.

🚀 **Q03** `CJ-003` • Basic

What features make Java a popular programming language?

⚡ **Quick answer**

Platform independence, automatic memory management, a strong type system, first-class concurrency, and the biggest backend ecosystem in the industry.

- **Platform independent**: bytecode runs on any JVM.
- **Object-oriented**: encapsulation, inheritance, polymorphism, abstraction.
- **Garbage collected**: G1 (default), ZGC and Shenandoah for low-latency workloads.
- **Robust**: compile-time type checking, checked exceptions, no pointer arithmetic.
- **Concurrent**: threads, `java.util.concurrent`, and virtual threads since Java 21.
- **Fast**: the JIT compiler optimises hot code using real runtime profiles.
- **Backward compatible**: code written decades ago still compiles and runs.

⚠️ **Interview trap**

Old answers list the **Security Manager** as a security feature. It was deprecated for removal in Java 17 and permanently disabled in JDK 24. Security today comes from bytecode verification, strong encapsulation of JDK internals, and the module system.

© TechTribers. For personal use only. Do not redistribute.

⚔️ Q04 CJ-004 • Basic

Compare C++ and Java: differences and similarities.

⚡ **Quick answer**

Both are statically typed and object-oriented with C-style syntax. Java trades low-level control (pointers, manual memory, multiple class inheritance) for safety and portability.

Aspect	C++	Java
Compilation	Native machine code per platform	Bytecode, run by the JVM
Memory	Manual (`new` / `delete` , RAII, smart pointers)	Garbage collector
Pointers	Raw pointers and pointer arithmetic	References only, no arithmetic
Multiple inheritance	Allowed for classes	Only through interfaces
Dynamic dispatch	Only methods marked `virtual`	Every instance method that is not `static` , `private` or `final`
Operator overloading	Supported	Not supported
Templates / generics	Templates, instantiated per type	Generics, erased at compile time

⚠️ **Interview trap**

"In Java all methods are virtual" is wrong. `static` , `private` and `final` methods are bound at compile time and cannot be overridden.

👋 Q05 CJ-005 • Basic

Write a simple "Hello, World!" program.

⚡ **Quick answer**

A public class with a `public static void main(String[] args)` method that calls `System.out.println` .

Compile and run:

- `javac HelloWorld.java` produces `HelloWorld.class`
- `java HelloWorld` starts the JVM and calls `main`
- Since Java 11 you can skip `javac` for single files: `java HelloWorld.java`

© TechTribers. For personal use only. Do not redistribute.

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

▶ Output

```
Hello, World!
```

🎯 **Likely follow-up**

Why is `main` static? Because the JVM calls it before any object of the class exists, so it must belong to the class, not an instance.

Q06 CJ-006 • Basic

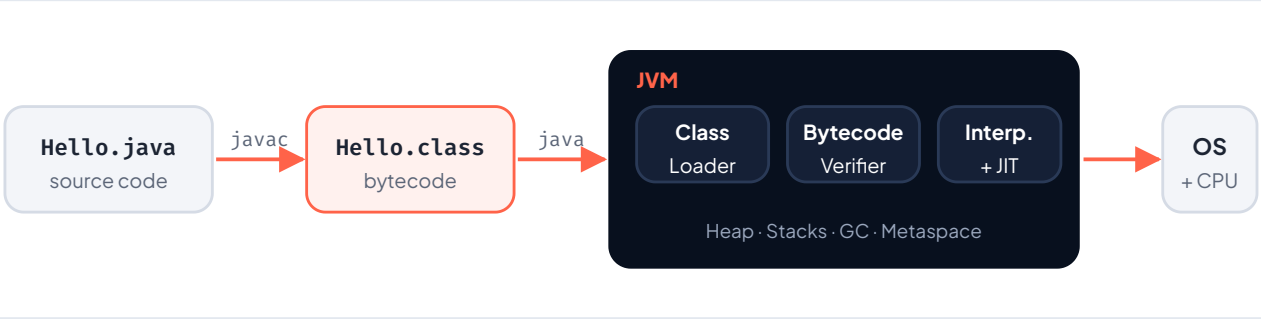
What is the structure of a Java program, and how does it run?

⚡ **Quick answer**

Code lives in classes, classes live in packages, and a `.java` file may contain at most one public top-level class whose name matches the file. Execution starts at `main`.

A typical source file is ordered like this: `package` declaration, then `import` statements, then type declarations (classes, interfaces, enums, records).

At runtime the **class loader** loads `.class` files on demand, the **bytecode verifier** checks them, and the **interpreter** runs them while the **JIT** compiles hot methods to native code.



© TechTribers. For personal use only. Do not redistribute.

```java
package com.techtribers.demo;       // 1. package

import java.time.LocalDate;          // 2. imports

public class Greeter {               // 3. one public class = file name
    public static void main(String[] args) {
        System.out.println("Today is " + LocalDate.now());
    }
}
```

🛠️ Q07 CJ-007 • Basic

How do you set the path for Java?

⚡ **Quick answer**

Point `JAVA_HOME` at the JDK installation directory and add `$JAVA_HOME/bin` to `PATH`.

```terminal
# Linux / macOS (~/.bashrc or ~/.zshrc)
export JAVA_HOME=/usr/lib/jvm/jdk-21
export PATH="$JAVA_HOME/bin:$PATH"

# Windows (PowerShell, persistent for the user)
setx JAVA_HOME "C:\Program Files\Java\jdk-21"
setx PATH "%JAVA_HOME%\bin;%PATH%"

# Verify
java -version
javac -version
```

💡 **Pro tip**

`PATH` lets the shell find `java` and `javac`. `JAVA_HOME` is what Maven, Gradle, IDEs and app servers read. In practice, use SDKMAN! (`sdk use java 21`) to switch versions.

© TechTribers. For personal use only. Do not redistribute.

Q08 CJ-008 • Basic

Differentiate between JDK, JRE and JVM.

> ⚡ **Quick answer**
> JVM runs bytecode. JRE = JVM + standard class libraries (enough to run). JDK = JRE + development tools like `javac`, `jshell`, `jar` (needed to build).

- **JVM** is a specification with many implementations (HotSpot, OpenJ9, GraalVM). It is platform-specific; that is exactly what makes bytecode platform-independent.
- **JRE** is what an end user needs to run an app.
- **JDK** is what a developer needs: compiler, debugger, packaging and diagnostic tools (`jcmd`, `jstack`, `jmap`).



> ⚠️ **Interview trap**
> Since Java 11, Oracle no longer ships a separate JRE. You build a trimmed custom runtime with `jlink`, or use a JDK-based container image.

Q09 CJ-009 • Intermediate

Describe the role and purpose of the JVM.

> ⚡ **Quick answer**
> The JVM loads, verifies and executes bytecode, manages memory and garbage collection, and abstracts the OS and CPU away from your code.

Main subsystems:

- **Class loader subsystem**: Bootstrap, Platform and Application loaders, with parent-first delegation.
- **Runtime data areas**: heap (objects), per-thread stacks (frames, local variables), Metaspace (class metadata, which replaced PermGen in Java 8), PC registers, native method stacks.
- **Execution engine**: interpreter, plus the C1 and C2 JIT compilers (tiered compilation), plus the garbage collector.

© TechTribers. For personal use only. Do not redistribute.

Its job is what makes Java safe and portable: no bytecode runs unverified, and no code touches raw memory.

🎯 **Likely follow-up**

What is tiered compilation? Code starts in the interpreter, then gets compiled by C1 (fast, light optimisation) once warm, then by C2 (aggressive optimisation) once hot, using profiling data gathered on the way.

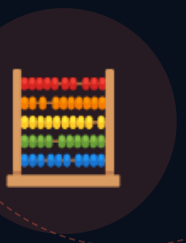
© TechTribers. For personal use only. Do not redistribute.

Section 02

Variables, Data Types & Operators

How Java stores values, what types exist, and the operators you are expected to know cold.

Questions 10–14 · 5 questions

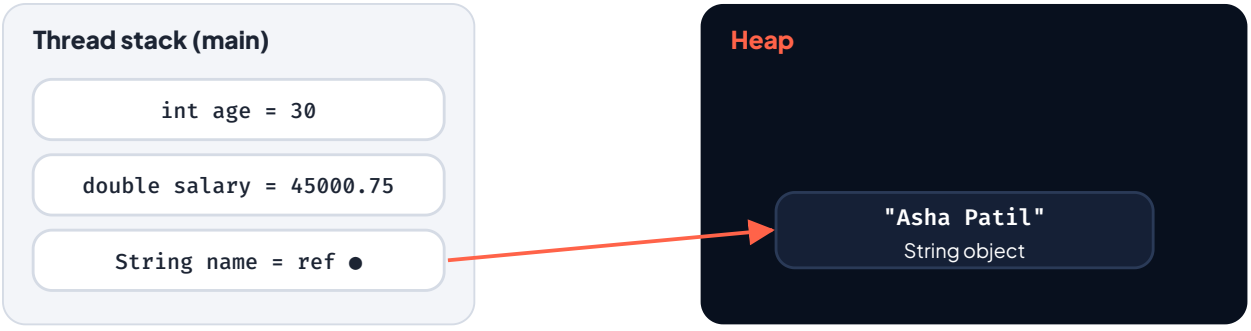


Q10 CJ-010 • Basic

What are variables in Java? How are they declared and used?

⚡ Quick answer

A variable is a named, typed storage slot. Java has local variables, instance fields, static fields and parameters, each with different scope and lifetime.



Primitives hold the value; references hold a pointer to a heap object.

Java

```java
public class VariablesExample {
    static int instances = 0;          // static: one per class
    String name;                       // instance: one per object

    public static void main(String[] args) {
        int age = 30;                  // local: lives on the stack frame
        double salary = 45_000.75;     // underscores for readability
        var city = "Pune";             // Java 10+: type inferred as String
        final int MAX_RETRIES = 3;     // cannot be reassigned

        System.out.println(age + " | " + salary + " | " + city);
    }
}
```

▶ Output

```
30 | 45000.75 | Pune
```


© TechTribers. For personal use only. Do not redistribute.

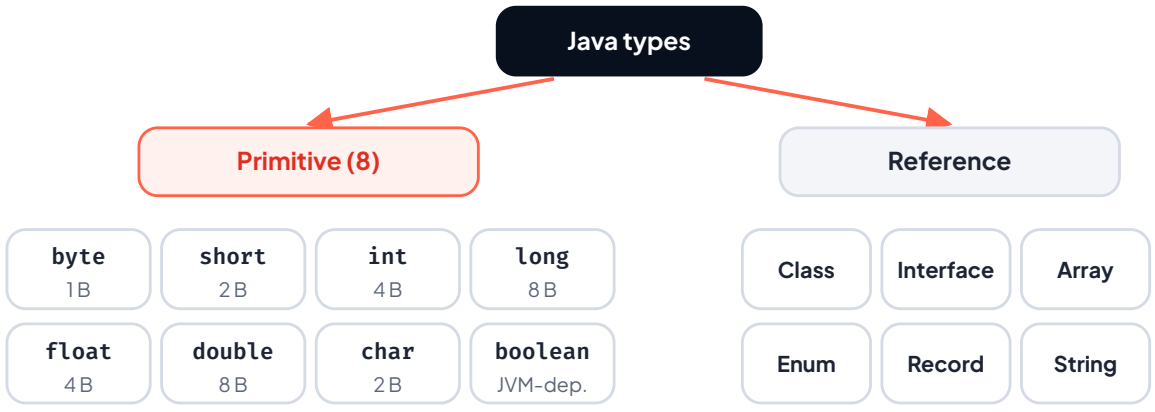
⚠️ **Interview trap**
Local variables get **no default value**. Reading one before assignment is a compile error. Fields do get defaults (`0` , `false` , `null`).

🔢 **Q11** `CJ-011` • Basic

Discuss the data types available in Java with examples.

⚡ **Quick answer**
Eight primitives (`byte` , `short` , `int` , `long` , `float` , `double` , `char` , `boolean`) that hold values directly, and reference types (classes, interfaces, arrays, enums, records) that hold a pointer to an object.

Every primitive has a **wrapper class** (`Integer` , `Long` , `Double` …). Autoboxing converts between them automatically. That is convenient, but it costs allocation in hot loops and can throw `NullPointerException` on unboxing.



© TechTribers. For personal use only. Do not redistribute.

```java
import java.util.Arrays;

public class DataTypesExample {
    public static void main(String[] args) {
        int age = 30;
        long population = 1_428_000_000L;   // L suffix required
        float ratio = 0.75f;                // f suffix required
        char grade = 'A';
        boolean employed = true;

        String name = "Asha";               // reference type
        int[] scores = {90, 85, 77};        // arrays are objects

        System.out.println(name + " " + Arrays.toString(scores));
    }
}
```

▶ Output

```
Asha [90, 85, 77]
```

⚠️ **Interview trap**

`Integer a = 127, b = 127; a == b` is `true`, but with `128` it is `false`. Values -128 to 127 come from the Integer cache; outside that range you get different objects. Always compare wrappers with `equals()`.

Q12 CJ-012 • Intermediate

How does Java handle Unicode characters, and why does it matter?

⚡ **Quick answer**

`String` and `char` are Unicode-based. A `char` is a 16-bit UTF-16 code unit, so characters outside the Basic Multilingual Plane (emoji, some CJK) take two `char`s.

Why it matters: internationalised apps (names in Devanagari, emoji in chat messages) break if you treat `length()` as "number of characters" or cut a string in the middle of a surrogate pair.

Since Java 18 the default charset is **UTF-8** everywhere (JEP 400). Since Java 9, *compact strings* store Latin-1–only strings in 1 byte per character internally.

© TechTribers. For personal use only. Do not redistribute.

```java
String s = "Hi 👋";
System.out.println(s.length());                    // 5, not 4
System.out.println(s.codePointCount(0, s.length()));   // 4 real characters
s.codePoints().forEach(cp -> System.out.print(Character.toChars(cp)));
```

▶ Output

```
5
4
Hi 👋
```

Q13 CJ-013 • Basic

Explain the types of operators in Java with examples.

⚡ Quick answer

Arithmetic, relational, logical, bitwise, shift, assignment, unary, ternary and `instanceof`. Know precedence, short-circuiting, and integer division.

```java
int a = 10, b = 3;

System.out.println(a / b);        // 3      integer division truncates
System.out.println(a % b);        // 1
System.out.println(a / (double) b); // 3.333...

boolean ok = (b != 0) && (a / b > 2); // && short-circuits: safe
System.out.println(a & b);        // 2      bitwise AND (1010 & 0011)
System.out.println(a ^ b);        // 9      XOR
System.out.println(a << 2);       // 40     multiply by 4
System.out.println(-16 >> 2);     // -4     sign-preserving shift
System.out.println(-16 >>> 28);   // 15     zero-fill shift

String label = a > b ? "bigger" : "smaller"; // ternary
int x = 5; x += 2.9;              // compound assignment casts: x = 7
```

⚠️ Interview trap

`&&` and `||` short-circuit; `&` and `|` on booleans evaluate **both** sides. Also, `x += 2.9` compiles on an `int` (implicit cast), but `x = x + 2.9` does not.

© TechTribers. For personal use only. Do not redistribute.

🔑 **Q14** `CJ-014` • Basic

List some important keywords in Java and their significance.

⚡ **Quick answer**

Keywords are reserved words that cannot be used as identifiers. Group them by purpose instead of reciting a list.

Group	Keywords
Types	`class` , `interface` , `enum` , `record` , *`extends`* , *`implements`*
Access	`public` , `protected` , `private`
Modifiers	`static` , `final` , `abstract` , `synchronized` , `volatile` , `transient` , `native` , `sealed` , `non-sealed`
Flow	`if` , `else` , `switch` , `case` , `default` , `for` , `while` , `do` , `break` , `continue` , `return` , `yield`
Exceptions	`try` , `catch` , `finally` , `throw` , `throws`
Objects	`new` , `this` , `super` , `instanceof`
Unused	`goto` , `const` (reserved but have no meaning)

* Contextual keywords: special only in specific positions, so you can still name a variable `record` or `yield` . `var` is a reserved type name, not a keyword.

🎯 **Likely follow-up**

Are `true` , `false` and `null` keywords? No. They are literals, but they are still reserved and cannot be used as identifiers.

© TechTribers. For personal use only. Do not redistribute.

Section 03

Control Flow

Branches, loops and jumps, plus the modern switch that interviewers now expect you to know.

Questions 15–22 · 8 questions



Q15 CJ-015

• Basic

What is the purpose of the if-else statement? Give an example.

⚡ Quick answer

It executes one block or another based on a boolean condition. Chain with `else if` for multiple branches.

Java

```
int score = 72;
String grade;
if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else if (score >= 60) {
    grade = "C";
} else {
    grade = "F";
}
System.out.println(grade);
```

▶ Output

```
C
```

💡 Pro tip

In real code prefer **guard clauses**: return early for invalid cases instead of nesting `if` blocks three levels deep. Reviewers love it.

© TechTribers. For personal use only. Do not redistribute.

Q16 CJ-016

• Basic

Explain the switch statement and provide an example.

⚡ Quick answer

`switch` selects a branch by matching a value. Since Java 14, switch **expressions** with `->` return a value and have no fall-through.

Allowed selector types: `int` and smaller integrals plus their wrappers, `char`, `String`, enums, and since Java 21 **any type** with pattern matching (`case Circle c ->`).

Java

```java
int day = 2;

// Classic statement: needs break, falls through without it
String name;
switch (day) {
    case 1: name = "Sunday"; break;
    case 2: name = "Monday"; break;
    default: name = "Invalid";
}

// Modern expression (Java 14+): no fall-through, must be exhaustive
String type = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> throw new IllegalArgumentException("day=" + day);
};
System.out.println(name + " / " + type);
```

▶ Output

```
Monday / Weekday
```

⚠️ Interview trap

Forgetting `break` in a classic switch silently runs the next case. This is a classic production bug and a classic interview question.

© TechTribers. For personal use only. Do not redistribute.

Q17 CJ-017

• Basic

What is the purpose of the for loop? Provide an example.

⚡ Quick answer

It repeats a block a known number of times using init, condition and update in one line. The enhanced for-each loop iterates arrays and any `Iterable`.

Java

```java
for (int i = 1; i <= 3; i++) {
    System.out.println("Iteration " + i);
}

List<String> langs = List.of("Java", "Kotlin", "Scala");
for (String lang : langs) {            // for-each
    System.out.println(lang);
}
```

▶ Output

```
Iteration 1
Iteration 2
Iteration 3
Java
Kotlin
Scala
```

⚠️ Interview trap

Removing elements from a `List` inside a for-each throws `ConcurrentModificationException`. Use `iterator.remove()` or `list.removeIf(...)`.

© TechTribers. For personal use only. Do not redistribute.

🔄 Q18 CJ-018 • Basic

Explain the while loop and provide an example.

⚡ Quick answer

`while` checks the condition **before** each iteration, so the body may run zero times. Use it when the number of iterations is not known upfront.

Java

```java
int attempts = 0;
boolean connected = false;
while (!connected && attempts < 3) {
    attempts++;
    connected = attempts == 2;        // pretend 2nd try succeeds
    System.out.println("Attempt " + attempts + " -> " + connected);
}
```

▶ Output

```
Attempt 1 -> false
Attempt 2 -> true
```

🔁 Q19 CJ-019 • Basic

Describe the do-while loop. Provide an example.

⚡ Quick answer

`do-while` checks the condition **after** the body, so the body always runs at least once.

Java

```java
Scanner in = new Scanner(System.in);
int choice;
do {
    System.out.println("1) Deposit  2) Withdraw  0) Exit");
    choice = in.nextInt();
} while (choice != 0);              // menu shown at least once
```

💡 Pro tip

The classic real-world use is a menu or a retry prompt: you always want to show it once before deciding whether to repeat.

© TechTribers. For personal use only. Do not redistribute.

Q20 CJ-020

• Basic

What is the purpose of the break statement? Give an example.

⚡ Quick answer

`break` exits the nearest loop or switch immediately. A **labelled** break exits an outer loop.

Java

```java
int[][] grid = {{1, 2}, {3, 42}, {5, 6}};
outer:
for (int[] row : grid) {
    for (int v : row) {
        if (v == 42) {
            System.out.println("Found 42");
            break outer;            // leaves BOTH loops
        }
    }
}
```

▶ Output

```
Found 42
```

Q21 CJ-021

• Basic

Explain the continue statement and provide an example.

⚡ Quick answer

`continue` skips the rest of the current iteration and jumps to the next one.

Java

```java
for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) continue;       // skip even numbers
    System.out.print(i + " ");
}
```

▶ Output

```
1 3 5
```


© TechTribers. For personal use only. Do not redistribute.

💬 **Q22** CJ-022 • Basic

Describe the different types of comments in Java.

⚡ **Quick answer**

Single-line `//`, multi-line `/* */`, and Javadoc `/* /`, which the `javadoc` tool turns into HTML API documentation.

Java

```java
/**
 * Calculates simple interest.
 * @param principal amount in INR
 * @param rate      annual rate, e.g. 7.5
 * @param years     duration
 * @return interest amount
 */
public static double interest(double principal, double rate, int years) {
    // rate is a percentage, convert to a fraction
    return principal * rate * years / 100;   /* SI formula */
}
```

💡 **Pro tip**

Good comments explain **why**, not what. Java 23 also added Markdown Javadoc comments using `///`.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Coding Programs

Small programs asked in screening rounds. Clean, correct, and with complexity stated.

Questions 23–24 · 2 questions

Q23 CJ-023

• Basic

Write a Java program to find the sum of all even numbers from 1 to 100.

⚡ Quick answer

Loop with a step of 2, use a stream, or use the arithmetic formula. Mention all three to show range.

Java

```java
public class SumOfEvens {
    public static void main(String[] args) {
        int loop = 0;
        for (int i = 2; i <= 100; i += 2) loop += i;          // O(n/2)

        int stream = java.util.stream.IntStream.rangeClosed(1, 100)
                .filter(n -> n % 2 == 0).sum();

        int n = 100 / 2;
        int formula = n * (n + 1);                            // O(1): 2+4+...+2n = n(n+1)

        System.out.println(loop + " " + stream + " " + formula);
    }
}
```

▶ Output

```
2550 2550 2550
```

Q24 CJ-024

• Basic

Write a program to check whether a number is a palindrome.

⚡ Quick answer

Reverse the digits mathematically and compare with the original. Negative numbers are not palindromes.

Complexity: **O(d)** time, where d is the number of digits, and O(1) space.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
public class PalindromeNumber {

    static boolean isPalindrome(int n) {
        if (n < 0) return false;
        int original = n, reversed = 0;
        while (n > 0) {
            reversed = reversed * 10 + n % 10;    // append last digit
            n /= 10;                              // drop last digit
        }
        return reversed == original;
    }

    public static void main(String[] args) {
        for (int n : new int[]{121, 123, 1001, -121}) {
            System.out.println(n + " -> " + isPalindrome(n));
        }
    }
}
```

▶ Output

```
121 -> true
123 -> false
1001 -> true
-121 -> false
```

🎯 **Likely follow-up**

Can `reversed` overflow? Yes, for values near `Integer.MAX_VALUE`. Fix it by reversing only half the digits and comparing halves, or by using `long`.

techtriber.com | Instagram @techtribers © TechTribers. For personal use only. Do not redistribute.

Section 05

Classes, Objects & OOP Pillars

The building blocks of every Java codebase and the four pillars you will be asked to explain with code.

Questions 25–35 · 11 questions



Q25 CJ-025

• Basic

What is a class?

⚡ Quick answer

A class is a blueprint that defines state (fields) and behaviour (methods). It is also a type, so the compiler can check how objects of it are used.

A class can contain fields, methods, constructors, initializer blocks, and nested types. Its metadata is loaded into Metaspace once, no matter how many objects you create.

Java

```
public class Car {
    private final String make;      // state
    private final String model;
    private int speed;

    public Car(String make, String model) {
        this.make = make;
        this.model = model;
    }

    public void accelerate(int delta) {   // behaviour
        speed += delta;
    }
}
```


© TechTribers. For personal use only. Do not redistribute.

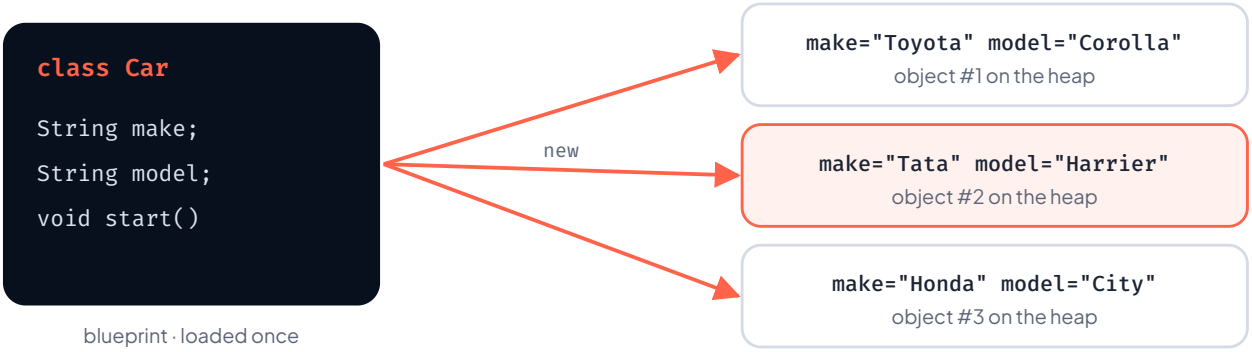
Q26 CJ-026 • Basic

What is an object?

⚡ Quick answer

An object is a runtime instance of a class, created with `new`, living on the heap, with its own copy of the instance fields.

Every object has three things: **state** (field values), **behaviour** (methods) and **identity** (it is a distinct object even if its state equals another's).



Java

```
Car c1 = new Car("Toyota", "Corolla");
Car c2 = new Car("Tata", "Harrier");
c1.accelerate(40);       // affects c1 only

Car c3 = c1;             // c3 is another REFERENCE to the same object
System.out.println(c1 == c3);   // true
```

▶ Output

```
true
```


© TechTribers. For personal use only. Do not redistribute.

Q27 CJ-027

• Basic

What is the difference between a class and an object?

⚡ Quick answer

A class is the definition, written once in source and loaded once. Objects are instances created from it at runtime, any number of times.

	Class	Object
Nature	Template / type	Instance of that type
Created	Declared in source, loaded by class loader	`new`, reflection, `clone()`, deserialization
Memory	Metadata in Metaspace	Heap, per instance
Count	One per class loader	Unlimited

🎯 Likely follow-up

Name four ways to create an object. `new`, `Class.getDeclaredConstructor().newInstance()`, `clone()`, and deserialization. Factory methods like `List.of()` still use one of these internally.

Q28 CJ-028

• Basic

What are instance variables?

⚡ Quick answer

Non-static fields declared in a class. Each object gets its own copy, stored inside the object on the heap, initialised to a default value if not set.

Java

```java
public class Student {
    String name;      // default null
    int age;          // default 0
    boolean active;   // default false
}

Student a = new Student();
a.name = "Asha";
Student b = new Student();
System.out.println(b.name + " " + b.age);
```

▶ Output

```
null 0
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Keep instance variables `private` and expose behaviour, not fields. That is encapsulation, and it is what the next few questions build on.

Q29 CJ-029 • Basic

What are instance methods?

> ⚡ **Quick answer**
> Non-static methods that run against a specific object. They can read and modify that object's fields through the implicit `this` reference.

Instance methods are dispatched dynamically: which implementation runs depends on the runtime class of the object, which is the basis of polymorphism.

Java

```java
public class Circle {
    private final double radius;
    public Circle(double radius) { this.radius = radius; }

    public double area() {                // instance method
        return Math.PI * radius * radius;
    }
}

System.out.printf("%.2f%n", new Circle(2).area());
```

▶ **Output**

```
12.57
```

Q30 CJ-030 • Basic

What is encapsulation?

> ⚡ **Quick answer**
> Bundling state with the behaviour that changes it, and hiding the state behind `private` so all changes go through methods that enforce the class's rules.

The benefit is **invariants**: the balance can never go negative or be set to garbage, because no outside code can touch the field directly.

© TechTribers. For personal use only. Do not redistribute.

```java
public class BankAccount {
    private BigDecimal balance = BigDecimal.ZERO;   // hidden

    public void deposit(BigDecimal amount) {
        if (amount.signum() <= 0)
            throw new IllegalArgumentException("amount must be positive");
        balance = balance.add(amount);
    }

    public BigDecimal getBalance() { return balance; }  // read-only view
}
```

⚠️ **Interview trap**

A class with private fields and a public getter **and** setter for each one is not really encapsulated. It is a struct with extra steps. Expose operations (`deposit`), not raw setters.

💡 **Pro tip**

Use `BigDecimal` for money, never `double` . `0.1 + 0.2` in `double` is `0.30000000000000004` .

Q31 CJ-031 • Basic

What is inheritance?

⚡ **Quick answer**

A subclass acquires the fields and methods of a superclass using `extends` , forming an "is-a" relationship. Java allows single inheritance of classes.

```java
class Animal {
    void sound() { System.out.println("Some sound"); }
}

class Dog extends Animal {          // Dog IS-A Animal
    @Override
    void sound() { System.out.println("Woof"); }
    void fetch() { System.out.println("Fetching"); }
}

Animal a = new Dog();
a.sound();        // Woof (runtime type decides)
// a.fetch();     // compile error: Animal has no fetch()
```

▶ **Output**

```
Woof
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> "Favour composition over inheritance." Use inheritance only for a true is-a relationship. Inheriting just to reuse code couples you to the parent's internals.

⬆️ **Q32** `CJ-032` • Basic

What is the `super` keyword used for?

> ⚡ **Quick answer**
> `super` refers to the parent class: `super(...)` calls a parent constructor, and `super.method()` / `super.field` access the parent's version of an overridden member.

* Java 25 (JEP 513, flexible constructor bodies) allows statements before `super(...)` as long as they do not touch `this`. On Java 21 and earlier, it must be first.

Java

```java
class Animal {
    protected final String type;
    Animal(String type) { this.type = type; }
    String describe() { return "Animal: " + type; }
}

class Dog extends Animal {
    private final String breed;
    Dog(String breed) {
        super("Mammal");            // must be the first statement*
        this.breed = breed;
    }
    @Override
    String describe() {
        return super.describe() + ", breed: " + breed;
    }
}

System.out.println(new Dog("Indie").describe());
```

▶ **Output**

```
Animal: Mammal, breed: Indie
```


© TechTribers. For personal use only. Do not redistribute.

Q33 CJ-033 • Basic

What is method overriding?

⚡ **Quick answer**

A subclass provides its own implementation of an inherited instance method with the same signature. The JVM picks the implementation at runtime based on the object's actual class.

Rules:

- Same name and parameter list; return type same or a **subtype** (covariant return).
- Access cannot be **more restrictive** (a `public` method cannot become `protected`).
- Cannot throw **new or broader checked** exceptions.
- `static`, `private` and `final` methods cannot be overridden.

Java

```
class Shape {
    double area() { return 0; }
}
class Square extends Shape {
    private final double side;
    Square(double side) { this.side = side; }
    @Override                       // compiler verifies it really overrides
    double area() { return side * side; }
}
Shape s = new Square(3);
System.out.println(s.area());
```

▶ **Output**

```
9.0
```

💡 **Pro tip**

Always use `@Override`. If you misspell the name or get a parameter type wrong, the compiler catches it instead of you silently creating an overload.

© TechTribers. For personal use only. Do not redistribute.

Q34 CJ-034

• Intermediate

Can you override a private or final method?

⚡ Quick answer

No. A `private` method is not visible to the subclass, so it cannot be overridden. A `final` method is explicitly locked against overriding.

Java

```java
class Parent {
    private void secret() { System.out.println("Parent secret"); }
    final void locked()   { System.out.println("Parent locked"); }
    void callSecret()     { secret(); }   // always calls Parent.secret
}
class Child extends Parent {
    private void secret() { System.out.println("Child secret"); } // new method, not an override
    // void locked() {}   // compile error: cannot override final
}
new Child().callSecret();
```

▶ Output

```
Parent secret
```

⚠️ Interview trap

Declaring a method with the same signature as a parent's `private` method compiles fine, which tricks people into thinking it overrides. It doesn't: adding `@Override` would fail to compile, and the parent keeps calling its own version.

© TechTribers. For personal use only. Do not redistribute.

Q35 CJ-035

• Basic

What is method overloading?

⚡ Quick answer

Multiple methods in the same class with the same name but different parameter lists. The compiler picks one at compile time from the argument types.

Java

```
class Calculator {
    int add(int a, int b)          { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int... nums)           { return java.util.Arrays.stream(nums).sum(); }
}
Calculator c = new Calculator();
System.out.println(c.add(2, 3));       // int version
System.out.println(c.add(2.5, 3.5));   // double version
System.out.println(c.add(1, 2, 3, 4)); // varargs
```

▶ Output

```
5
6.0
10
```

⚠️ Interview trap

Changing only the **return type** is not overloading; it is a compile error. Resolution order also matters: exact match, then widening, then boxing, then varargs.

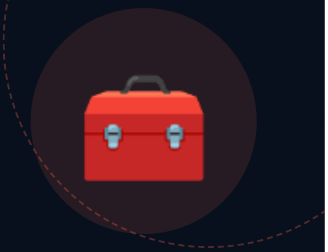
© TechTribers. For personal use only. Do not redistribute.

Section 06

Constructors & Object Basics

How objects are born, how `this` works, and how to copy and type-check them safely.

Questions 36–41 · 6 questions



Q36 CJ-036 • Basic

What is a constructor?

⚡ **Quick answer**

A special block with the class's name and no return type, run by `new` to put a freshly allocated object into a valid initial state.

Constructors are not inherited and cannot be `static`, `final` or `abstract`. They can be `private`, which is how singletons and static factory classes stop outside instantiation.

Java

```
public class Car {
    private final String make;
    private final String model;

    public Car(String make, String model) {
        this.make = Objects.requireNonNull(make);
        this.model = Objects.requireNonNull(model);
    }
}
```

💡 **Pro tip**

Validate inputs in the constructor. An object that cannot be created invalid never needs to be checked for validity later.

© TechTribers. For personal use only. Do not redistribute.

Q37 CJ-037

• Basic

Can you have multiple constructors in a class?

⚡ **Quick answer**

Yes, through constructor overloading. Chain them with `this(...)` so the initialisation logic lives in one place.

Java

```java
public class Car {
    private final String make;
    private final String model;

    public Car(String make, String model) {    // the "full" constructor
        this.make = make;
        this.model = model;
    }

    public Car(String make) {
        this(make, "Unknown");                 // delegate, don't duplicate
    }
}
```

💡 **Pro tip**

Once you have more than three or four optional parameters, switch to the **Builder** pattern instead of piling up constructors (the "telescoping constructor" anti-pattern).

Q38 CJ-038

• Basic

What is the default constructor?

⚡ **Quick answer**

If a class declares **no** constructor, the compiler adds a public no-argument one that just calls `super()`. Declare any constructor and the default disappears.

Java

```java
class A { }                 // compiler adds: A() { super(); }

class B {
    B(int x) { }            // now there is NO B()
}

new A();                    // OK
// new B();                 // compile error
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

This bites with frameworks. JPA/Hibernate entities and Jackson DTOs need a no-arg constructor. Add a parameterised one and forget the no-arg, and it breaks at runtime, not at compile time.

👉 **Q39** `CJ-039` • Basic

What is the purpose of the `this` keyword?

⚡ **Quick answer**

`this` is a reference to the current object. Use it to disambiguate fields from parameters, chain constructors with `this(...)`, pass the current object, or return it for fluent APIs.

Java

```java
public class Person {
    private String name;
    private int age;

    public Person(String name) {
        this(name, 0);                // constructor chaining
    }
    public Person(String name, int age) {
        this.name = name;             // field vs parameter
        this.age = age;
    }
    public Person withAge(int age) {
        this.age = age;
        return this;                  // fluent API
    }
}
```


© TechTribers. For personal use only. Do not redistribute.

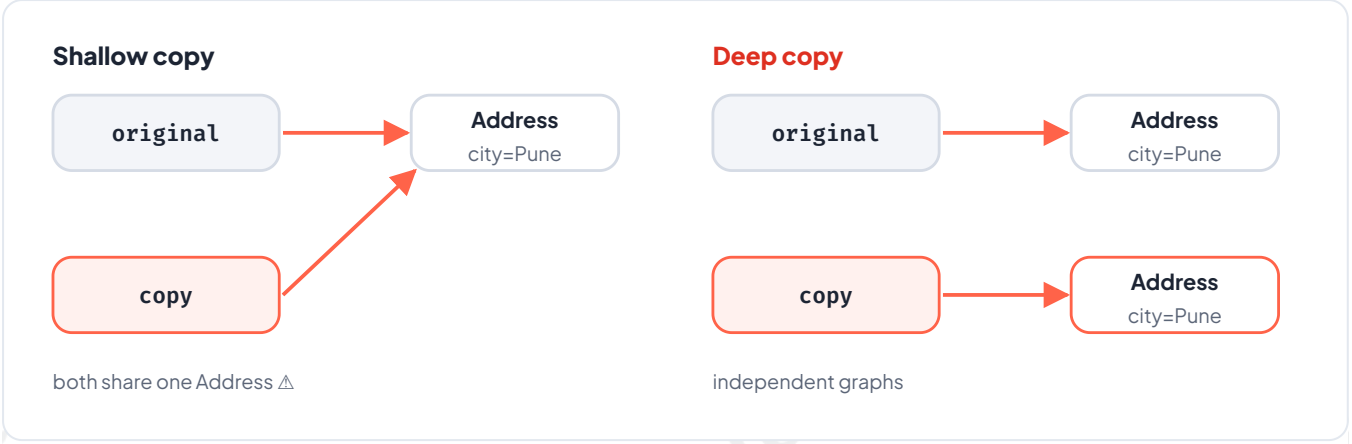
Q40 CJ-040

• Intermediate

What is the difference between a shallow copy and a deep copy?

⚡ **Quick answer**

A shallow copy duplicates the top-level object but shares referenced objects. A deep copy duplicates the whole object graph, so the copy and original are fully independent.



Java

```
record Address(String city) { }              // immutable
class Employee {
    String name; List<String> skills;
    Employee(String name, List<String> skills) { this.name = name; this.skills = skills; }

    Employee shallowCopy() { return new Employee(name, skills); }
    Employee deepCopy()    { return new Employee(name, new ArrayList<>(skills)); }
}

Employee e1 = new Employee("Asha", new ArrayList<>(List.of("Java")));
Employee s = e1.shallowCopy();
Employee d = e1.deepCopy();
e1.skills.add("Kafka");
System.out.println(s.skills + " " + d.skills);
```

▶ **Output**

```
[Java, Kafka] [Java]
```

💡 **Pro tip**

Immutable objects (`String` , records with immutable fields) never need deep copying. They can be shared safely. That is one of the strongest arguments for immutability.

© TechTribers. For personal use only. Do not redistribute.

🔍 Q41 CJ-041

• Basic

What is the `instanceof` operator used for?

⚡ **Quick answer**

It checks whether an object is an instance of a class or interface (including subtypes). Since Java 16, pattern matching tests and casts in one step.

Java

```java
Object obj = "TechTribers";

// Old style: check, then cast
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Java 16+: pattern matching
if (obj instanceof String s && !s.isBlank()) {
    System.out.println(s.toUpperCase());
}

System.out.println(null instanceof Object);   // always false
```

▶ **Output**

```
11
TECHTRIBERS
false
```

💡 **Pro tip**

Lots of `instanceof` chains is a design smell. Prefer polymorphism, or since Java 21, a `switch` with type patterns over a sealed hierarchy.

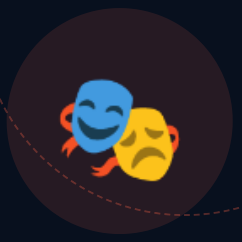
© TechTribers. For personal use only. Do not redistribute.

Section 07

Polymorphism, Abstraction & Interfaces

The questions that separate memorised definitions from real design understanding.

Questions 42–50 · 9 questions



Q42 CJ-042

• Basic

What is polymorphism?

⚡ Quick answer

One interface, many implementations. A reference of a parent type can point to any subtype, and the right method is chosen at runtime.

Two kinds:

- **Compile-time (static)**: method overloading. The compiler picks the method from argument types.
- **Runtime (dynamic)**: method overriding. The JVM picks the implementation from the object's actual class (dynamic dispatch through the vtable).

Java

```
interface PaymentMethod { String pay(int amount); }
class Upi  implements PaymentMethod { public String pay(int a) { return "UPI ₹" + a; } }
class Card implements PaymentMethod { public String pay(int a) { return "Card ₹" + a; } }

List<PaymentMethod> methods = List.of(new Upi(), new Card());
for (PaymentMethod m : methods) {
    System.out.println(m.pay(500));   // no if/else on type
}
```

▶ Output

```
UPI ₹500
Card ₹500
```

💡 Pro tip

The interview-winning point: polymorphism lets you add a new payment type **without touching** the calling code. That is the Open/Closed principle in action.

© TechTribers. For personal use only. Do not redistribute.

Q43 CJ-043 • Basic

What is an abstract class?

⚡ Quick answer

A class marked `abstract` that cannot be instantiated. It can mix abstract methods (no body) with concrete methods, fields and constructors, giving subclasses a partial implementation.

This is the **Template Method** pattern: the abstract class owns the algorithm (`describe`) and subclasses fill in the variable step (`area`).

Java

```java
abstract class Shape {
    private final String name;
    protected Shape(String name) { this.name = name; }

    abstract double area();                  // subclasses must implement

    public String describe() {               // shared concrete logic
        return "%s with area %.2f".formatted(name, area());
    }
}

class Circle extends Shape {
    private final double r;
    Circle(double r) { super("Circle"); this.r = r; }
    double area() { return Math.PI * r * r; }
}

System.out.println(new Circle(1).describe());
```

▶ Output

```
Circle with area 3.14
```


© TechTribers. For personal use only. Do not redistribute.

Q44 CJ-044

• Intermediate

Can an abstract class have a constructor?

⚡ Quick answer

Yes. You cannot call it with `new`, but subclass constructors call it through `super(...)` to initialise the abstract class's own fields.

Java

```
abstract class Animal {
    protected final String name;
    Animal(String name) {
        this.name = name;
        System.out.println("Animal() for " + name);
    }
    abstract void sound();
}

class Dog extends Animal {
    Dog(String name) { super(name); }
    void sound() { System.out.println(name + ": Woof"); }
}

new Dog("Bruno").sound();
```

▶ Output

```
Animal() for Bruno
Bruno: Woof
```

⚠️ Interview trap

Never call an overridable (abstract) method from a constructor. It runs the subclass's override **before** the subclass's fields are initialised, so you see `null` or `0`.

Q45 CJ-045

• Basic

What is an interface?

⚡ Quick answer

A contract: a type that declares what an implementing class can do. A class can implement many interfaces, which is how Java gets multiple inheritance of type.

Since Java 8 interfaces can hold `default` and `static` methods, and since Java 9 `private` methods. An interface with exactly one abstract method is a **functional interface** and can be implemented with a lambda.

© TechTribers. For personal use only. Do not redistribute.

```java
public interface Notifier {
    void send(String to, String message);          // abstract (implicitly public)

    default void sendAll(List<String> to, String msg) {   // Java 8+
        to.forEach(t -> send(t, msg));
    }

    static Notifier console() {                    // Java 8+
        return (to, msg) -> System.out.println(to + " <- " + msg);
    }
}

Notifier.console().sendAll(List.of("asha", "ravi"), "Build passed");
```

▶ Output

```
asha <- Build passed
ravi <- Build passed
```

Q46 CJ-046 • Intermediate

What is the difference between an abstract class and an interface?

⚡ Quick answer

An abstract class is a partial implementation with state, for closely related classes. An interface is a capability contract with no instance state, which any class can adopt alongside others.

	Abstract class	Interface
Instance fields	Yes, any	No; only `public static final` constants
Constructors	Yes	No
Method types	Abstract and concrete, any access	Abstract, `default`, `static`, `private` (9+)
Inheritance	A class extends one	A class implements many
Relationship	"is-a" (Dog is an Animal)	"can-do" (Dog is Comparable)
Use when	Sharing state and code across a family	Defining a role many unrelated classes can play

⚠️ Interview trap

"An interface can only have abstract methods" has been false since Java 8. Saying it in 2026 instantly dates your preparation material.

© TechTribers. For personal use only. Do not redistribute.

Q47 CJ-047

• Basic

Can an interface have variables?

⚡ Quick answer

Only constants. Every field in an interface is implicitly `public static final` and must be initialised.

Java

```
interface Limits {
    int MAX_RETRIES = 3;   // really: public static final int MAX_RETRIES = 3;
}
System.out.println(Limits.MAX_RETRIES);
// Limits.MAX_RETRIES = 5;  // compile error
```

▶ Output

```
3
```

⚠️ Interview trap

The "constant interface" anti-pattern (implementing an interface just to inherit its constants) leaks those constants into your class's public API. Use a `final` class with a private constructor, or an `enum`.

Q48 CJ-048

• Intermediate

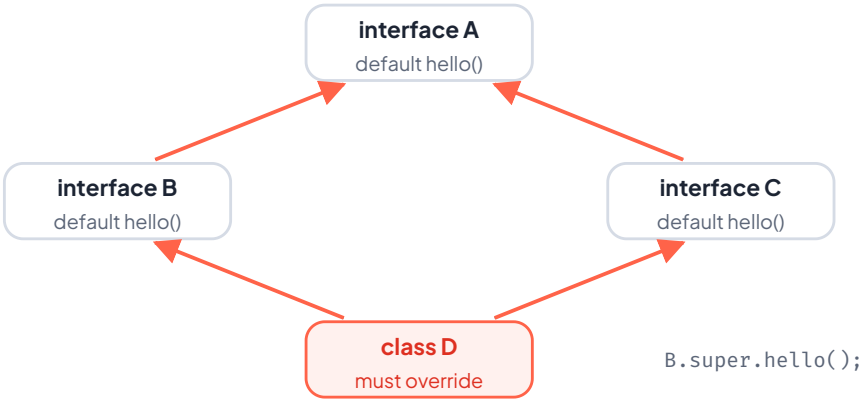
What is a default method in an interface?

⚡ Quick answer

A method with a body in an interface, marked `default`. It lets you add methods to an existing interface without breaking every class that already implements it.

This is why Java 8 could add `stream()` and `forEach()` to `Collection` without breaking every collection library in existence. Resolution rules: class methods win over interface defaults; a more specific interface wins over a less specific one; otherwise you must override.

© TechTribers. For personal use only. Do not redistribute.



Java

```java
interface Greeter  { default String hello() { return "Hello"; } }
interface Welcomer { default String hello() { return "Welcome"; } }

class Reception implements Greeter, Welcomer {
    @Override
    public String hello() {                // conflict MUST be resolved
        return Greeter.super.hello() + " & " + Welcomer.super.hello();
    }
}
System.out.println(new Reception().hello());
```

▶ Output

```
Hello & Welcome
```

techtriber.com

© TechTribers. For personal use only. Do not redistribute.

Q49 CJ-049

• Basic

Can an interface extend another interface?

⚡ Quick answer

Yes, with `extends`, and an interface can extend **multiple** interfaces. The implementing class must satisfy all of them.

Java

```java
interface Readable { String read(); }
interface Writable { void write(String data); }
interface ReadWrite extends Readable, Writable { }   // multiple

class MemoryStore implements ReadWrite {
    private String data = "";
    public String read() { return data; }
    public void write(String d) { data = d; }
}
```

Q50 CJ-050

• Intermediate

What is a static method in an interface?

⚡ Quick answer

A method declared `static` in an interface. It belongs to the interface itself, is called as `InterfaceName.method()`, and is **not inherited** by implementing classes.

Java

```java
interface MathOps {
    static int add(int a, int b) { return a + b; }
}
class Impl implements MathOps { }

System.out.println(MathOps.add(2, 3));   // OK
// Impl.add(2, 3);                       // compile error: not inherited
```

▶ Output

```
5
```

💡 Pro tip

Real examples: `List.of()`, `Map.of()`, `Comparator.comparing()`. They are factory and utility methods that belong with the type they create.

© TechTribers. For personal use only. Do not redistribute.

Section 08

Nested & Inner Classes

Four kinds of classes inside classes, and the memory-leak trap that comes with one of them.

Questions 51–55 · 5 questions



Q51 CJ-051 • Basic

What is a nested class?

⚡ **Quick answer**

A class declared inside another class. Static ones are "static nested classes"; non-static ones are "inner classes" (member, local and anonymous).

Use nested classes when a type only makes sense together with its outer class, such as `Map.Entry`, a `Builder`, or a `Node` inside a linked list.

Static nested	Inner (member)	Local	Anonymous
`static class B`	`class B`	`class B inside a method`	`new I() { … }`
no outer instance	holds Outer.this	method scope only	one-off, no name

Java

```java
public class Outer {
    private int x = 10;

    static class StaticNested { }   // no link to an Outer instance
    class Inner {                   // has a hidden Outer.this reference
        int read() { return x; }
    }
}
```


© TechTribers. For personal use only. Do not redistribute.

Q52 CJ-052

Intermediate

What is a static nested class?

⚡ Quick answer

A nested class marked `static`. It does not need an outer instance and can only access the outer class's static members directly.

Java

```
public class Pizza {
    private final String size;
    private final List<String> toppings;

    private Pizza(Builder b) { size = b.size; toppings = b.toppings; }

    public static class Builder {          // classic use: Builder pattern
        private String size = "M";
        private final List<String> toppings = new ArrayList<>();
        public Builder size(String s) { size = s; return this; }
        public Builder add(String t) { toppings.add(t); return this; }
        public Pizza build() { return new Pizza(this); }
    }
}

Pizza p = new Pizza.Builder().size("L").add("paneer").build();
```

💡 Pro tip

Default to `static` for nested classes. Only drop it when the nested class genuinely needs the outer instance.

© TechTribers. For personal use only. Do not redistribute.

Q53 CJ-053

• Intermediate

What is a local inner class?

⚡ Quick answer

A class declared inside a method or block. It is visible only within that block and can use the method's local variables if they are effectively final.

Java

```
void printReport(List<Integer> values, int threshold) {
    class Stats {                                   // local class
        long above() {
            return values.stream().filter(v -> v > threshold).count();
        }
    }
    System.out.println("Above " + threshold + ": " + new Stats().above());
}
```

⚠️ Interview trap

Captured local variables must be **effectively final**. If you reassign `threshold` anywhere in the method, the code stops compiling.

Q54 CJ-054

• Intermediate

What is an anonymous inner class?

⚡ Quick answer

A class with no name, declared and instantiated in one expression, usually to implement an interface or extend a class on the spot.

For functional interfaces, lambdas replaced anonymous classes. Anonymous classes still matter when you need state, multiple methods, or to extend an abstract class.

© TechTribers. For personal use only. Do not redistribute.

```java
Comparator<String> byLength = new Comparator<>() {   // anonymous class
    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
};

Comparator<String> lambda = Comparator.comparingInt(String::length); // modern

List<String> names = new ArrayList<>(List.of("Ravindra", "Asha", "Om"));
names.sort(lambda);
System.out.println(names);
```

▶ Output

```
[Om, Asha, Ravindra]
```

🎯 **Likely follow-up**

What does `this` mean inside an anonymous class versus a lambda? In an anonymous class, `this` is the anonymous object. In a lambda, `this` is the enclosing instance.

🎁 Q55 CJ-055 • Intermediate

What is an inner class, and what are its advantages?

⚡ **Quick answer**

A non-static nested class. Each instance is tied to an outer instance and can access all the outer object's members, including private ones.

Advantages:

- **Encapsulation**: the iterator can read private state without exposing it.
- **Organisation**: helper types stay next to the only code that uses them.
- **Natural fit** for iterators, event listeners and callbacks.

© TechTribers. For personal use only. Do not redistribute.

```java
public class Playlist {
    private final List<String> songs = new ArrayList<>();
    public void add(String s) { songs.add(s); }

    public Iterator<String> shuffled() { return new ShuffleIterator(); }

    private class ShuffleIterator implements Iterator<String> {   // inner
        private final List<String> order = new ArrayList<>(songs); // reads outer's private field
        { Collections.shuffle(order); }
        public boolean hasNext() { return !order.isEmpty(); }
        public String next() { return order.remove(0); }
    }
}
```

⚠️ **Interview trap**

Every inner instance holds a strong reference to its outer object. Hand an inner object to a long-lived cache or listener registry and the outer object can never be garbage collected. This is a well-known memory-leak source, especially on Android.

© TechTribers. For personal use only. Do not redistribute.

Section 09

final, static, this & super

Modifiers that look trivial and generate a surprising number of tricky follow-up questions.

Questions 56–66 · 11 questions



Q56 CJ-056

• Basic

What is a final class?

⚡ Quick answer

A class declared `final` cannot be extended. Use it to guarantee behaviour and immutability can't be altered by a subclass.

JDK examples: `String`, `Integer` and the other wrappers, `LocalDate`. Records are implicitly final.

Java

```java
public final class Money {              // nobody can subclass and break it
    private final long paise;
    public Money(long paise) { this.paise = paise; }
    public Money plus(Money o) { return new Money(paise + o.paise); }
}
// class FakeMoney extends Money { }   // compile error
```


© TechTribers. For personal use only. Do not redistribute.

Q57 CJ-057

• Basic

What is a final method?

⚡ Quick answer

A method declared `final` cannot be overridden by subclasses. It is still inherited and callable.

Java

```
abstract class PaymentProcessor {
    public final void process(int amount) {   // the algorithm is locked
        validate(amount);
        charge(amount);
        audit(amount);
    }
    protected abstract void charge(int amount);  // the step that varies
    private void validate(int a) { if (a <= 0) throw new IllegalArgumentException(); }
    private void audit(int a) { System.out.println("audited " + a); }
}
```

💡 Pro tip

Template Method pattern again: make the skeleton `final` so no subclass can skip validation or auditing.

Q58 CJ-058

• Basic

What is a final variable?

⚡ Quick answer

A variable that can be assigned exactly once. For primitives the value is fixed; for references the **reference** is fixed, not the object it points to.

Java

```
final int max = 3;
// max = 4;                        // compile error

final List<String> list = new ArrayList<>();
list.add("still mutable");         // fine: the object can change
// list = new ArrayList<>();       // compile error: the reference can't

class Circle {
    static final double PI = 3.14159;   // constant
    private final double radius;        // "blank final": set in constructor
    Circle(double r) { radius = r; }
}
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`final` does not make an object immutable. For a truly read-only list use `List.of(...)` or `Collections.unmodifiableList(...)`.

📊 **Q59** CJ-059 • Basic

What is a static variable?

⚡ **Quick answer**

A field that belongs to the class, not to any object. There is exactly one copy, shared by all instances, created when the class is initialised.

Java

```java
class Counter {
    static int created = 0;     // shared
    final int id;               // per object
    Counter() { id = ++created; }
}
new Counter(); new Counter(); Counter c = new Counter();
System.out.println(c.id + " of " + Counter.created);
```

▶ **Output**

```
3 of 3
```

⚠️ **Interview trap**

Mutable static state is shared across **all threads**. `++created` is not atomic, so under concurrency you lose updates. Use `AtomicInteger` or avoid mutable statics entirely.

© TechTribers. For personal use only. Do not redistribute.

Q60 CJ-060

• Basic

What is the `final` keyword used for?

⚡ Quick answer

`final` means "cannot change": a final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.

Applied to	Effect	Typical reason
Variable	Assigned once	Constants, immutability, capture in lambdas
Method	Cannot be overridden	Protect an algorithm or invariant
Class	Cannot be extended	Immutable value types, security
Parameter	Cannot be reassigned in the method	Readability

🎯 Likely follow-up

Why does `final` matter for concurrency? The Java Memory Model guarantees that `final` fields set in a constructor are visible to every thread once the constructor finishes, without synchronisation. This is why immutable objects are thread-safe.

Q61 CJ-061

• Basic

What is the `static` keyword used for?

⚡ Quick answer

`static` makes a member belong to the class instead of an instance: static fields, static methods, static nested classes, static initializer blocks, and static imports.

Java

```java
public final class MathUtil {
    private MathUtil() { }                    // no instances
    public static final int SCALE = 2;        // constant
    public static int square(int x) { return x * x; }
}

import static java.lang.Math.max;             // static import
int big = max(MathUtil.square(4), 10);
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
A static method cannot use `this` or access instance members directly; there is no object. Static methods also cannot be overridden: a subclass can only **hide** them.

🔗 Q62 CJ-062 • Intermediate

Can `this` be used in a static context, and what is constructor chaining?

⚡ **Quick answer**
No: static code has no current object, so `this` does not exist there. Constructor chaining is calling one constructor from another with `this(...)`, which must be the first statement.

Java

```java
public class Connection {
    private final String host; private final int port; private final int timeoutMs;

    public Connection(String host) { this(host, 5432); }
    public Connection(String host, int port) { this(host, port, 3_000); }
    public Connection(String host, int port, int timeoutMs) {
        this.host = host; this.port = port; this.timeoutMs = timeoutMs;
    }

    static void util() {
        // System.out.println(this.host);   // compile error: no 'this' in static
    }
}
```

⚠️ **Interview trap**
A constructor can call `this(...)` **or** `super(...)`, never both. If it calls `this(...)`, the target constructor is the one that ends up calling `super(...)`.

🪜 Q63 CJ-063 • Intermediate

What are the rules for calling `super(...)` in a constructor?

⚡ **Quick answer**
Every constructor calls a parent constructor. If you don't write `super(...)` or `this(...)`, the compiler inserts `super()`, which fails if the parent has no no-arg constructor.

Construction always runs **top-down**: `Object`, then each ancestor, then the class itself. On Java 25+ you may put validation statements before `super(...)` as long as they don't use `this` (JEP 513).

© TechTribers. For personal use only. Do not redistribute.

```java
class Vehicle {
    Vehicle(String id) { System.out.println("Vehicle " + id); }   // no no-arg ctor
}

class Truck extends Vehicle {
    Truck() {
        super("TRK-01");      // required: implicit super() would not compile
        System.out.println("Truck ready");
    }
}
new Truck();
```

▶ Output

```
Vehicle TRK-01
Truck ready
```

Q64 CJ-064 • Intermediate

How do you prevent a class from being inherited?

⚡ Quick answer

Make it `final`. Alternatives: make all constructors `private` (subclasses can't call `super`), or since Java 17 use `sealed` to allow only a named set of subclasses.

```java
public final class Rupee { }                        // 1. no subclasses at all

public class Registry {                             // 2. private constructor
    private Registry() { }
    public static Registry create() { return new Registry(); }
}

public sealed interface Shape permits Circle, Square { }   // 3. controlled set
public record Circle(double r) implements Shape { }
public record Square(double side) implements Shape { }
```

💡 Pro tip

Sealed types plus a pattern-matching `switch` give you exhaustiveness checks: if someone adds a new `Shape`, every `switch` that forgot to handle it fails to compile.

© TechTribers. For personal use only. Do not redistribute.

Q65 CJ-065

• Intermediate

Which methods cannot be overridden, and how do you prevent overriding?

⚡ Quick answer

Mark a method `final` to prevent overriding. `private` and `static` methods can't be overridden either: private ones are invisible, and static ones can only be hidden.

Method	Overridable?	What a same-signature method in a subclass does
`final`	No	Compile error
`private`	No	Declares an unrelated new method
`static`	No	Hides the parent's method (resolved by reference type)
Constructor	No	Not inherited in the first place
Regular instance method	Yes	Overrides (resolved by runtime type)

Q66 CJ-066

• Basic

Can a class be both `abstract` and `final`?

⚡ Quick answer

No, it is a compile error. `abstract` means "must be subclassed to be useful" and `final` means "can never be subclassed". They contradict each other.

The same applies to methods: `abstract` cannot be combined with `final`, `private` or `static`.

Java

```java
// abstract final class Broken { }    // error: illegal combination of modifiers

final class Utils {                    // what people actually want:
    private Utils() { }                // non-instantiable AND non-extendable
    static int twice(int x) { return x * 2; }
}
```


© TechTribers. For personal use only. Do not redistribute.

Section 10

Object Methods & Initializers

The `java.lang.Object` contract every class inherits, and the exact order in which an object gets initialised.

Questions 67–72 · 6 questions



Q67 CJ-067

• Basic

What is the purpose of the `toString()` method?

⚡ Quick answer

It returns a human-readable representation of an object. The default `ClassName@hexHashCode` is useless, so override it for logging and debugging.

Java

```
class Person {
    private final String name; private final int age;
    Person(String n, int a) { name = n; age = a; }
    @Override
    public String toString() { return "Person[name=%s, age=%d]".formatted(name, age); }
}

record PersonRecord(String name, int age) { }   // toString generated for you

System.out.println(new Person("Asha", 30));
System.out.println(new PersonRecord("Ravi", 28));
```

▶ Output

```
Person[name=Asha, age=30]
PersonRecord[name=Ravi, age=28]
```

⚠️ Interview trap

Never include passwords, tokens or full card numbers in `toString()`. Objects end up in logs far more often than you expect.

© TechTribers. For personal use only. Do not redistribute.

Q68 CJ-068

• Intermediate

What is the `equals()` method used for?

> ⚡ **Quick answer**
> `equals()` checks **logical** equality (same meaningful state), whereas `==` checks **reference** equality (same object). The default `equals()` is just `==`.

The contract: **reflexive**, **symmetric**, **transitive**, **consistent**, and `x.equals(null)` is `false`.

Java

```
class Person {
    private final String name; private final int age;
    Person(String n, int a) { name = n; age = a; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Person p)) return false;   // pattern matching
        return age == p.age && Objects.equals(name, p.name);
    }
    @Override
    public int hashCode() { return Objects.hash(name, age); }   // ALWAYS together
}

Person a = new Person("Asha", 30), b = new Person("Asha", 30);
System.out.println((a == b) + " " + a.equals(b));
```

▶ Output

```
false true
```

Q69 CJ-069

• Intermediate

What is the `hashCode()` method used for?

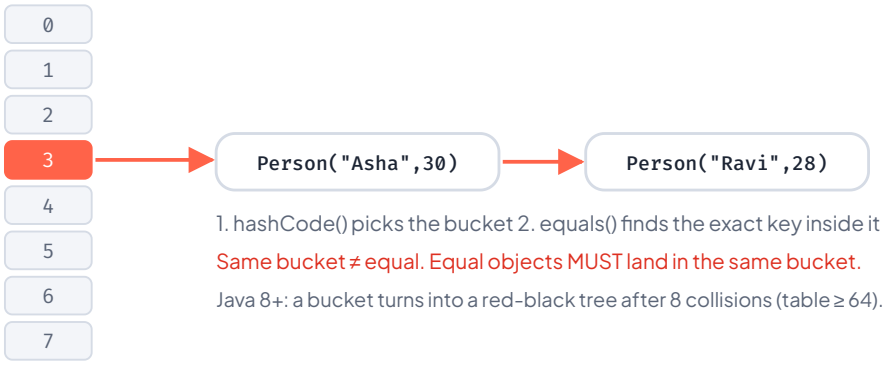
> ⚡ **Quick answer**
> `hashCode()` returns an `int` used by hash-based collections (`HashMap`, `HashSet`) to pick a bucket. It is **not** unique: different objects can share a hash code.

The contract with `equals()`:

- If `a.equals(b)`, then `a.hashCode() == b.hashCode()`. **Mandatory.**
- If hash codes are equal, the objects may or may not be equal (a collision).
- The hash code must not change while the object is in a hash collection.

© TechTribers. For personal use only. Do not redistribute.

`map.put(key, v) → hash(key.hashCode()) & (n-1) → bucket index`



1. hashCode() picks the bucket 2. equals() finds the exact key inside it

Same bucket ≠ equal. Equal objects MUST land in the same bucket.

Java 8+: a bucket turns into a red-black tree after 8 collisions (table ≥ 64).

Java

```java
class Key {
    final String id;
    Key(String id) { this.id = id; }
    @Override public boolean equals(Object o) { return o instanceof Key k && k.id.equals(id); }
    // hashCode NOT overridden -> broken
}
Map<Key, String> m = new HashMap<>();
m.put(new Key("A1"), "value");
System.out.println(m.get(new Key("A1")));
```

▶ Output

```
null
```

⚠️ **Interview trap**

Override `equals()` without `hashCode()` and the map lookup above returns `null` : the two equal keys land in different buckets. Also never use mutable fields in `hashCode()` for objects stored as keys.

© TechTribers. For personal use only. Do not redistribute.

Q70 CJ-070

• Advanced

What is the purpose of the `clone()` method?

⚡ **Quick answer**

`Object.clone()` makes a field-by-field (shallow) copy, but only if the class implements the `Cloneable` marker interface; otherwise it throws `CloneNotSupportedException`.

Java

```
class Team implements Cloneable {
    String name; List<String> members = new ArrayList<>();

    @Override
    public Team clone() {                          // covariant return, no checked exception
        try {
            Team t = (Team) super.clone();         // shallow copy
            t.members = new ArrayList<>(members); // fix up mutable fields manually
            return t;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);           // can't happen: we are Cloneable
        }
    }
}

// Preferred alternative: a copy constructor
// Team(Team other) { this.name = other.name; this.members = new ArrayList<>(other.members); }
```

💡 **Pro tip**

`clone()` is widely considered broken (Effective Java, Item 13): it bypasses constructors, fights with `final` fields, and is shallow by default. Prefer **copy constructors** or static copy factories, and say so in the interview.

© TechTribers. For personal use only. Do not redistribute.

Q71 CJ-071 • Intermediate

What is the instance initializer block used for?

⚡ Quick answer

A `{ }` block in a class body that runs every time an object is created: after the `super(...)` call, before the rest of the constructor body, together with field initialisers in source order.

1. **static fields + static {} blocks**
   once, at class initialisation (parent first)
2. **super(...) runs**
   the whole parent chain is built first
3. **instance fields + instance {} blocks**
   in textual order
4. **constructor body**
   the code after super(...)

Java

```java
class Demo {
    int x = log("field x");
    { log("instance block"); }
    Demo() { super(); log("constructor body"); }
    static int log(String s) { System.out.println(s); return 1; }
}
new Demo();
```

▶ Output

```
field x
instance block
constructor body
```

⚠️ Interview trap

"It runs before the constructor" is imprecise and loses marks. It runs **after** the superclass constructor finishes, and before the remaining statements of this constructor.

© TechTribers. For personal use only. Do not redistribute.

Q72 CJ-072 • Intermediate

What is the static initializer block used for?

⚡ Quick answer

A `static { }` block that runs exactly once, when the class is initialised (first active use), and is used for static setup that needs more than a one-line initialiser.

Java

```
class Config {
    static final Map<String, String> DEFAULTS;
    static {
        System.out.println("Loading Config once");
        Map<String, String> m = new HashMap<>();
        m.put("region", "ap-south-1");
        m.put("timeout", "3000");
        DEFAULTS = Collections.unmodifiableMap(m);
    }
}
System.out.println(Config.DEFAULTS.get("region"));
System.out.println(Config.DEFAULTS.get("timeout"));
```

▶ Output

```
Loading Config once
ap-south-1
3000
```

⚠️ Interview trap

If a static block throws, you get `ExceptionInInitializerError` on first use, and every later use of the class fails with `NoClassDefFoundError`. Painful to debug in production, so keep static blocks simple.

© TechTribers. For personal use only. Do not redistribute.

Section 11

Inheritance Edge Cases

Multiple inheritance and method hiding: short questions with a high rate of wrong answers.

Questions 73–74 · 2 questions

Q73 CJ-073

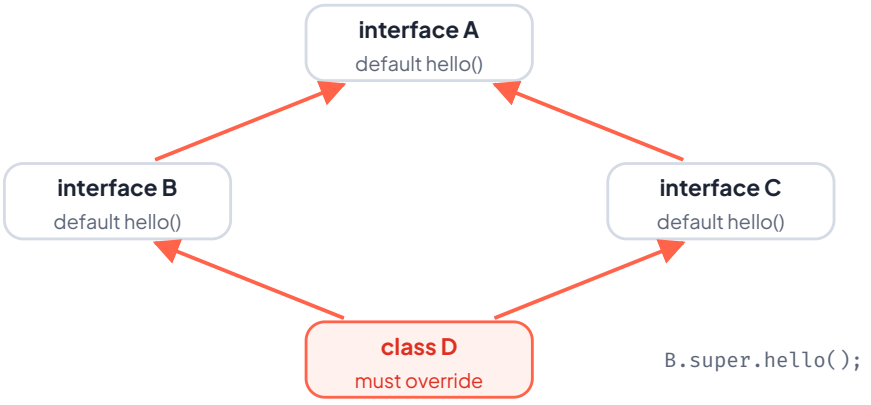
• Intermediate

How do you achieve multiple inheritance in Java?

⚡ Quick answer

Java has no multiple inheritance of **state** (one superclass only), but it supports multiple inheritance of **type and behaviour** through interfaces with default methods.

Classes are restricted to one parent to avoid the **diamond problem**: if two parents had the same field or method implementation, which one would the child inherit? With interfaces, the compiler forces you to resolve conflicts explicitly.



© TechTribers. For personal use only. Do not redistribute.

```java
interface Flyer   { default String move() { return "fly"; } }
interface Swimmer { default String move() { return "swim"; } }

class Duck implements Flyer, Swimmer {
    @Override
    public String move() {                      // diamond conflict: must resolve
        return Flyer.super.move() + " + " + Swimmer.super.move();
    }
}
System.out.println(new Duck().move());
```

▶ Output

```
fly + swim
```

Q74 CJ-074 • Intermediate

What is method hiding in Java?

⚡ Quick answer

When a subclass declares a `static` method with the same signature as a parent's static method, it **hides** it. The method is chosen by the **reference type** at compile time, not by the object.

```java
class Parent {
    static String who()   { return "Parent.static"; }
    String instance()     { return "Parent.instance"; }
}
class Child extends Parent {
    static String who()   { return "Child.static"; }      // hiding
    @Override String instance() { return "Child.instance"; } // overriding
}

Parent p = new Child();
System.out.println(p.who());       // reference type wins
System.out.println(p.instance());  // runtime type wins
```

▶ Output

```
Parent.static
Child.instance
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Calling a static method through an instance (`p.who()`) compiles but misleads readers. IDEs warn about it. Always call static methods as `ClassName.method()` .

techtriber.com

© TechTribers. For personal use only. Do not redistribute.